

PART 3: COMBINATIONAL LOGIC

MULTIPLIER & COMPARATOR

SAAD BAIG – DIGITAL LOGIC DESIGN



The animations and contents in these slides are originally adapted from the DLD course slides designed by Dr. Hasan Baig and Mr. Junaid Ahmed Memon. The original slides can be accessed from <https://www.hasanbaig.com/resources>

© Hasan Baig and Junaid Ahmed Memon. All rights reserved. These files and accompanying materials are made available under the terms of “Attribution-NonCommercial-NoDerivatives 4.0 International (CC BY-NC-ND 4.0) License” and is available at <https://creativecommons.org/licenses/by-nc-nd/4.0/>

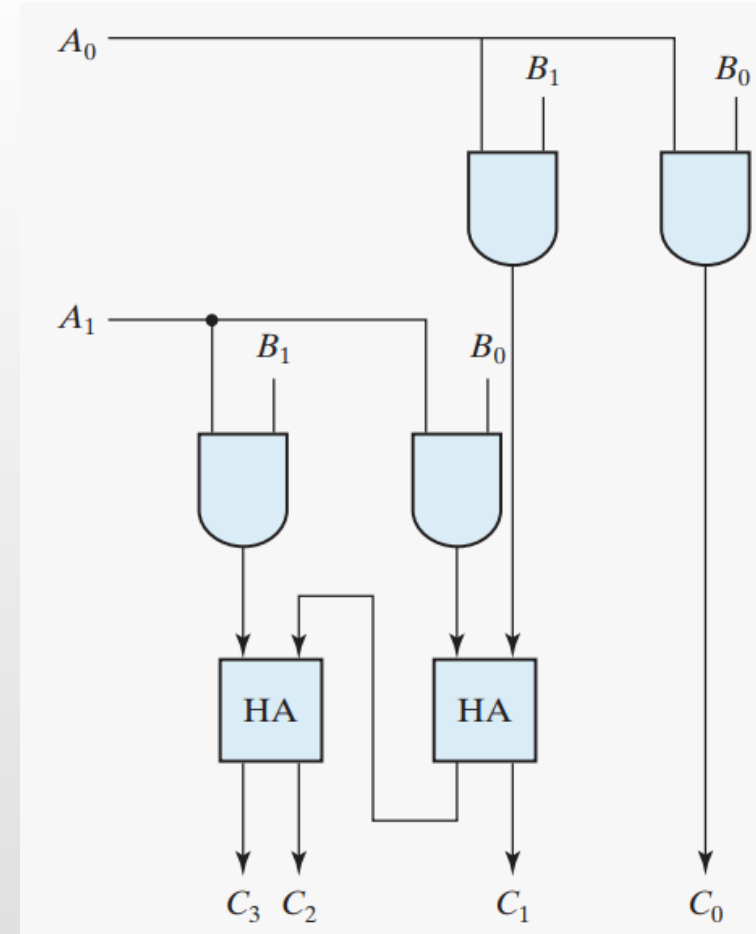
BINARY MULTIPLAYER



- Multiplication of binary numbers is performed in the same way as multiplication of decimal numbers.
- The multiplicand is multiplied by each bit of the multiplier, starting from the least significant bit.
- Each such multiplication forms a partial product. Successive partial products are shifted one position to the left.
- The final product is obtained from the sum of the partial products.

		B_1	B_0
	A_1	A_0B_1	A_0B_0
	A_1B_1	A_1B_0	
C_3	C_2	C_1	C_0

- Note that the least significant bit of the product does not have to go through an adder, since it is formed by the output of the first AND gate



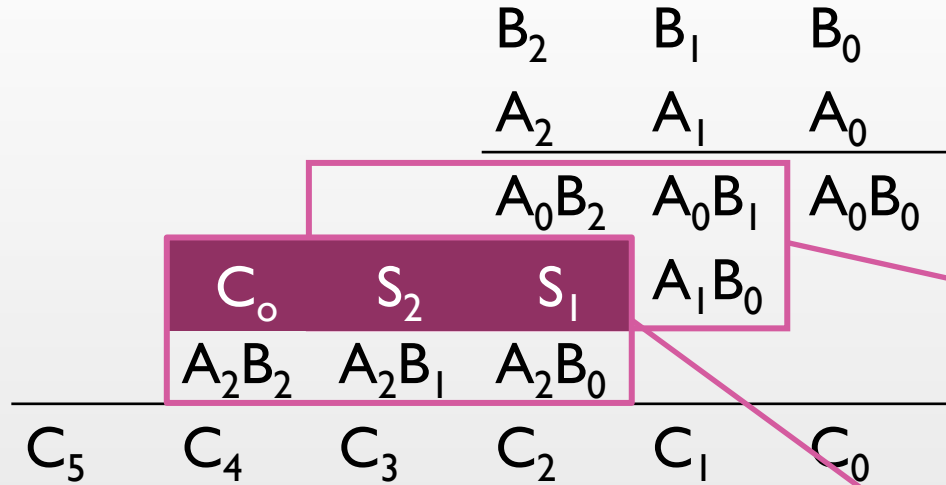
2-BIT BY 2-BIT BINARY MULTIPLIER



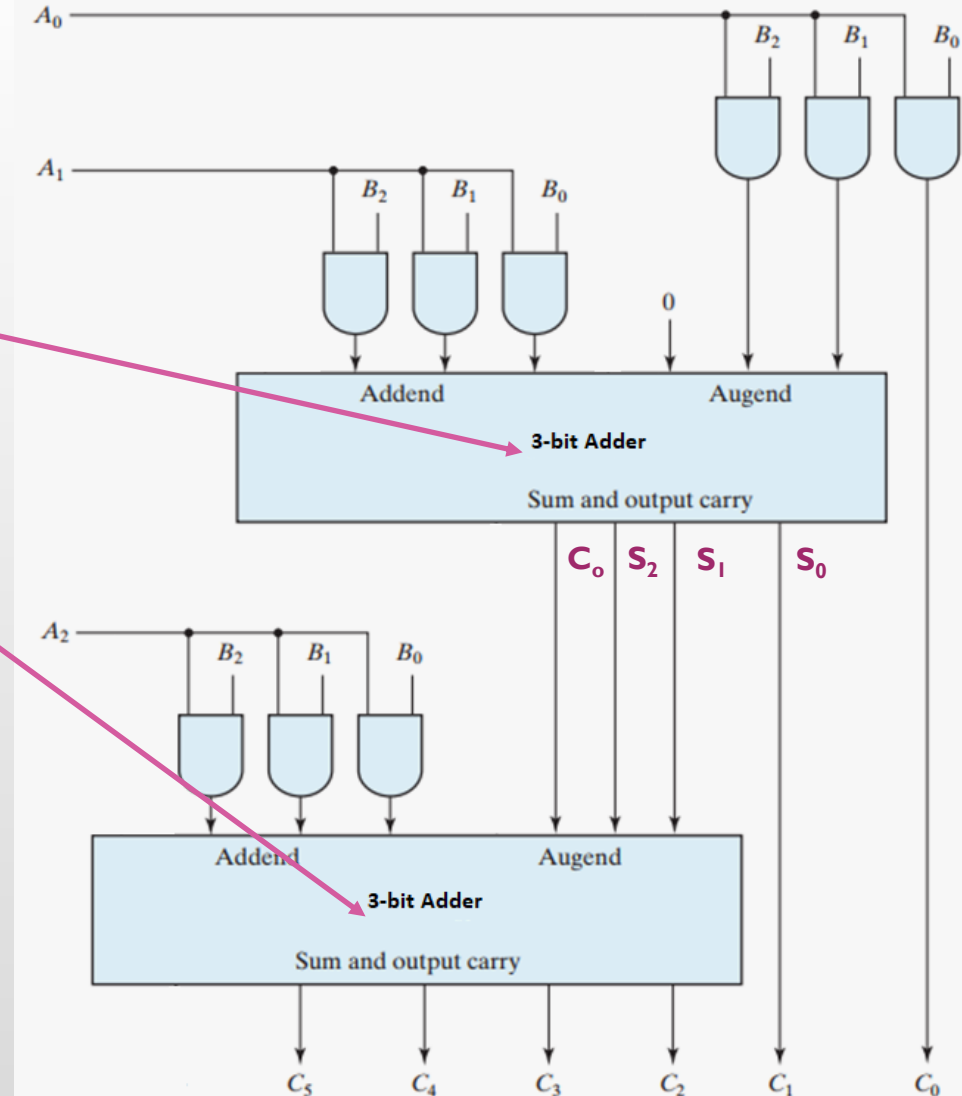
3-BIT BY 3-BIT BINARY MULTIPLIER

- Let's design 3-bit by 3-bit multiplier i.e. $B_2B_1B_0 \times A_2A_1A_0$. Assume you have 3-bit adder available.

			B_2	B_1	B_0
			A_2	A_1	A_0
			A_0B_2	A_0B_1	A_0B_0
		A_1B_2	A_1B_1	A_1B_0	
	A_2B_2	A_2B_1	A_2B_0		
C_5	C_4	C_3	C_2	C_1	C_0

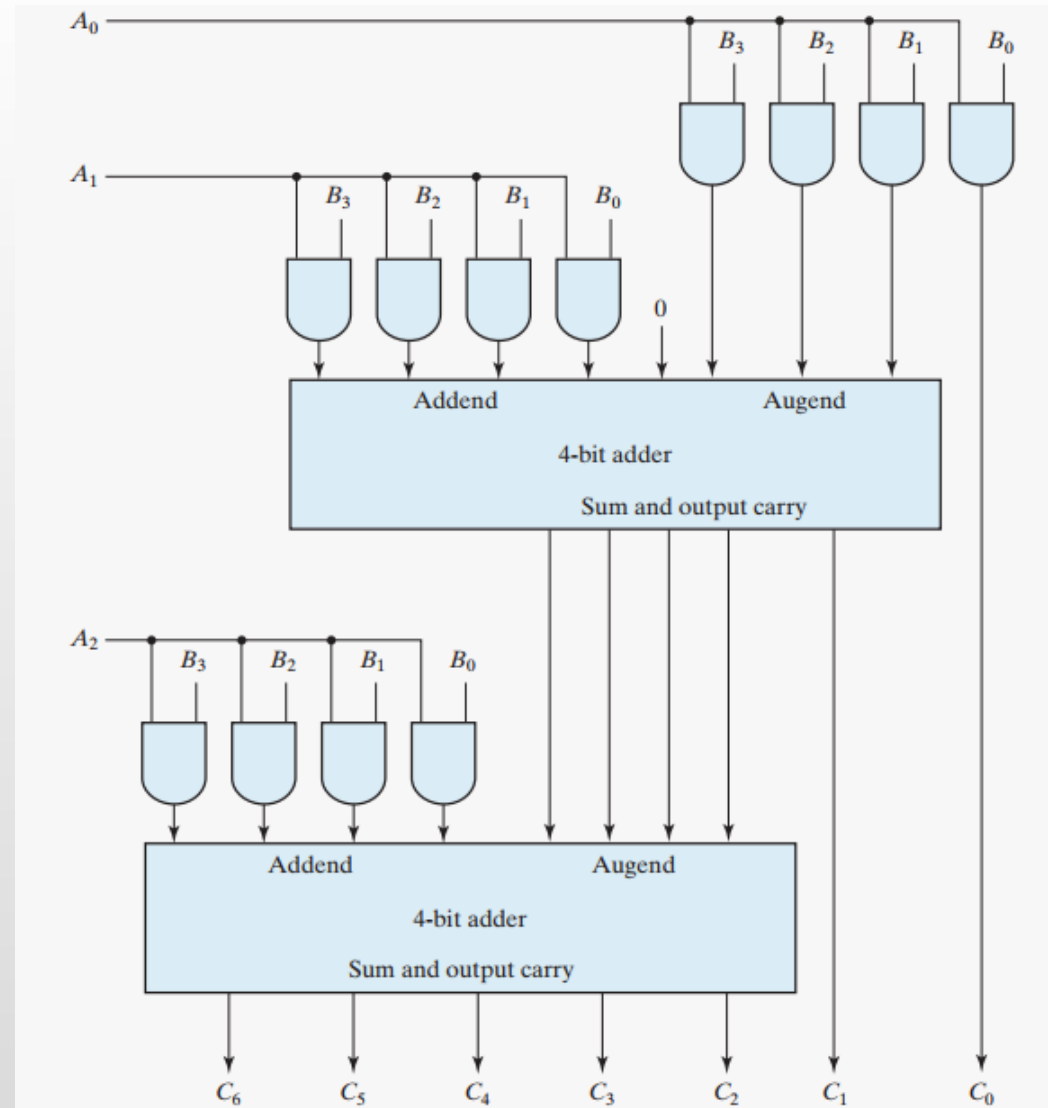


For A multiplier bits and B multiplicand bits, we need $(A \times B)$ AND gates and $(A - 1)$ B -bit adders to produce a product of $(A + B)$ bits.



- **Example:** 4-bit into 3-bit multiplier
- $B_3B_2B_1B_0 \times A_2A_1A_0$

For A multiplier bits and B multiplicand bits, we need $(A \times B)$ AND gates and $(A - 1)$ B-bit adders to produce a product of $(A + B)$ bits.





MAGNITUDE COMPARATOR

- A magnitude comparator is a combinational circuit that compares two numbers A and B and determines their relative magnitudes.
- The outcome of the comparison is specified by three binary variables that indicate whether $A > B$, $A = B$, or $A < B$.
- Circuit design by truth table for two n-bit comparator will require 2^{2n} rows in truth table.
 - **Example:** For 4-bit comparator there will be truth table of 256 rows!



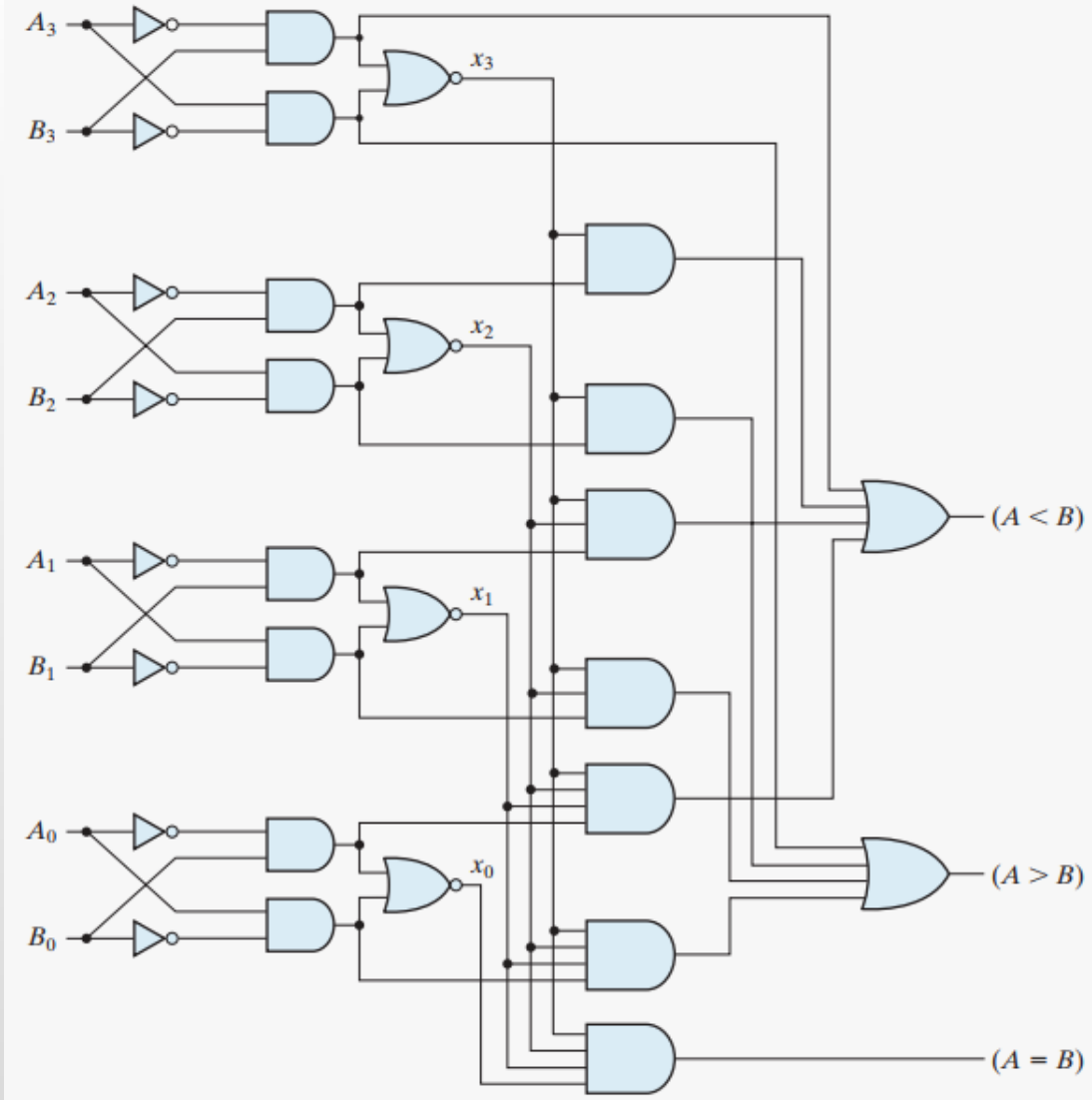
MAGNITUDE COMPARATOR

- Design Approach: Deriving an algorithm for the design of a four-bit magnitude comparator.
- Comparing two numbers $A = A_3A_2A_1A_0$ and $B = B_3B_2B_1B_0$, we get three results:
 - **Equality:** $A = B$
 - **Inequality:** $A > B$ or $A < B$
- We can check each number bitwise, and see if they match or not.
- For equality ($A = B$), what logic corresponds to it?
 - If both A_i and B_i are 0, or both are 1, output should be high.
 - Equality check $\rightarrow X = A_iB_i + A'_iB'_i$



MAGNITUDE COMPARATOR

- To determine whether A is greater or less than B , we inspect the relative magnitudes of pairs of significant digits, starting from the most significant position.
- If the two digits of a pair are equal, we compare the next lower significant pair of digits. The comparison continues until a pair of unequal digits is reached.
- If the corresponding digit of A is 1 and that of B is 0, we conclude that $A > B$. If the corresponding digit of A is 0 and that of B is 1, we have $A < B$.
- Inequality check $\rightarrow X = A_i B'_i + A'_i B_i$ (?)



$$(A < B) = A'_3B_3 + x_3A'_2B_2 + x_3x_2A'_1B_1 + x_3x_2x_1A'B_0$$

$$(A > B) = A_3B'_3 + x_3A_2B'_2 + x_3x_2A_1B'_1 + x_3x_2x_1A_0B'_0$$

MAGNITUDE COMPARATOR